MESS MANAGEMENT SYSTEM

Enterprise-Grade Technical Architecture & Functional Specification Blueprint

Document Version: 2.0.0-PROD

Target Audience: Full-Stack Software Engineers, System Architects, DevOps

Architecture Pattern: Microservices / Monolith Layered MVC

Date: June 2026

Status: Approved for Development

TABLE OF CONTENTS

1. Executive Summary & High-Level Core Requirements
2. System Architecture & High-Availability Technology Stack
3. Database Schema Design & Relational Data Integrity
4. Role-Based Access Control (RBAC) & Authentication Logic
5. Dynamic Bazaar Scheduling Priority Algorithm
6. Meal Management Lifecycle & Strict Cut-Off Window Constraints
7. Automated Financial Ledger Engine & Monthly Balancing Formulation
8. Integrated Third-Party Services (Payment Gateways & Messaging Channels)
9. Comprehensive Database Migration & Seeding Strategies
10. Security Constraints, Failover Paradigms, and System Audit Logs

1. EXECUTIVE SUMMARY

The Mess Management System (MMS) is engineered to automate and digitize shared residential food and communal finance setups, natively referred to as a "Mess" or shared student hostel. Managing these ecosystems manually consistently introduces accounting inconsistencies, disputes regarding dynamic price fluctuations, and transparency deadlocks across recurring manager rotations.

This enterprise software architecture aims to resolve tracking vulnerabilities through a rigorous data model that locks financial computations behind decentralized ledgers, enforce automated algorithmic assignments for utility workflows (such as market shopping distributions), and eliminate analytical human error during monthly account balance evaluations.

2. SYSTEM ARCHITECTURE & TECH STACK

To fulfill constraints including scaling efficiency, real-time sync states, and high data atomicity, the system adopts a decoupled, client-server model layout:

2.1 Server Architecture Layer

The backend engine is engineered via **Node.js (TypeScript/Express)** or **Python (FastAPI)**. This layer manages validation, token parsing, scheduled billing events, and interface pipelines with external service entities.

2.2 Persistent & Cache Database Layer

- **PostgreSQL (Primary DB):** Utilized exclusively to guarantee ACID properties over financial entries, ensuring strict transaction ledgers through strict constraint validations.
- **Redis Cache Engine:** Employed to buffer volatile operations like transient real-time meal states and token session lists, keeping primary resource requirements minimal.

2.3 User Interface Layout Layer

Crafted with a mobile-first orientation using **React Native** / **Flutter** or a responsive web assembly via **Tailwind CSS**, allowing swift daily logs by residents on standard handheld viewports.

> **Architecture Core Philosophy:** No direct modifications to transactional histories are permitted. Financial status fields adjust implicitly by computing aggregate balance mutations over immutable source logs.

3. DETAILED DATABASE SCHEMA DESIGN

The schema enforces strict data normalization down to 3NF standards, mapping user accounts, food logs, cash ledgers, and inventory balances.

3.1 Users Database Schema Table

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(120) UNIQUE NOT NULL,
    phone VARCHAR(20) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(30) NOT NULL DEFAULT 'member', -- admin, manager, member
    status VARCHAR(20) NOT NULL DEFAULT 'active', -- active, suspended, moved_out
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

3.2 Daily Meal Records Table

```
CREATE TABLE meals (
    id SERIAL PRIMARY KEY,
    user_id INT REFERENCES users(id) ON DELETE RESTRICT,
    date DATE NOT NULL,
    breakfast NUMERIC(3,1) DEFAULT 0.0, -- supports fractions e.g., 0.5, 1.0, 1.5
    lunch NUMERIC(3,1) DEFAULT 0.0,
    dinner NUMERIC(3,1) DEFAULT 0.0,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_by INT REFERENCES users(id),
    UNIQUE(user_id, date)
);
```

3. DATABASE SCHEMA DESIGN (CONTINUED)

3.3 Cash Deposits Ledger Table

```
CREATE TABLE deposits (
    id SERIAL PRIMARY KEY,
    user_id INT REFERENCES users(id) ON DELETE RESTRICT,
    received_by INT REFERENCES users(id) ON DELETE RESTRICT, -- Active manager ID
    amount NUMERIC(12, 2) NOT NULL CHECK (amount > 0),
    payment_method VARCHAR(30) NOT NULL, -- cash, bkash, nagad, upi
    transaction_reference VARCHAR(100) UNIQUE,
    is_cleared BOOLEAN DEFAULT FALSE,
    cleared_at TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

3.4 Daily Bazaar Outlays Log Table

```
CREATE TABLE bazaar_logs (
    id SERIAL PRIMARY KEY,
    buyer_id INT REFERENCES users(id) ON DELETE RESTRICT,
    authorized_by INT REFERENCES users(id) ON DELETE RESTRICT,
    date DATE NOT NULL,
    breakdown_notes TEXT NOT NULL, -- JSON structured breakdown of items bought
    total_cost NUMERIC(12, 2) NOT NULL CHECK (total_cost >= 0),
    invoice_img_url VARCHAR(255),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

4. ROLE-BASED ACCESS CONTROL & PERMISSION GRID

Security patterns require strict decoupling of management capabilities from base resident profiles. This system isolates operations across three explicitly bounded roles:

Capability Endpoint / Action	Super Admin	Current Manager	General Member
Configure Room Pricing / Penalties	Allowed	Denied	Denied
Approve Inbound Cash Deposits	Allowed	Allowed	Denied
Inject New Daily Bazaar Costs	Allowed	Allowed	Denied
Toggle Personal Next-Day Meals	Allowed	Allowed	Allowed
Modify External Peers' Meal States	Allowed	Allowed	Denied

4.1 Mid-Route Middleware Implementation Logic

Every execution lifecycle hitting a mutation path passes through an active token inspection block verifying expiration parameters along with assigned system tier privileges:

```
// Pseudo-code middleware enforcement pattern
function verifyRole(allowedRoles) {
    return (req, res, next) => {
        const userRole = req.tokenPayload.role;
        if (!allowedRoles.includes(userRole)) {
            return res.status(403).json({ error: "Access denied. Insufficient
privileges." });
        }
        next();
    };
}
```

5. AUTOMATED BAZAAR SCHEDULING PRIORITY QUEUE ENGINE

To eliminate administrative human bias or regular avoidance behaviors regarding tracking marketplace duties, the system enforces an algorithmic task distribution queue based on personal contribution ratios.

5.1 The Mathematical Priority Equation

The scheduling ranking logic calculates an absolute value score. The member possessing the largest positive priority gap is automatically marked for top billing to lead upcoming purchasing rounds:

```
Priority Score (S) = Total Approved Deposits (D) - Total Executed
Outlays (B)
```

5.2 SQL Production Pipeline Execution Implementation

The backend engine targets this automated scheduling distribution dynamically via a comprehensive tracking database abstraction block:

```
SELECT
    u.id AS member_id,
    u.name AS member_name,
    COALESCE(SUM(d.amount) FILTER (WHERE d.is_cleared = TRUE), 0) AS
total_deposited,
    COALESCE(SUM(b.total_cost), 0) AS total_bazaared,
    (COALESCE(SUM(d.amount) FILTER (WHERE d.is_cleared = TRUE), 0) -
COALESCE(SUM(b.total_cost), 0)) AS priority_score
FROM users u
LEFT JOIN deposits d ON u.id = d.user_id
LEFT JOIN bazaar_logs b ON u.id = b.buyer_id
WHERE u.status = 'active'
GROUP BY u.id, u.name
ORDER BY priority_score DESC, u.id ASC;
```

6. MEAL MANAGEMENT LIFECYCLE & LOCKOUT CONSTRAINTS

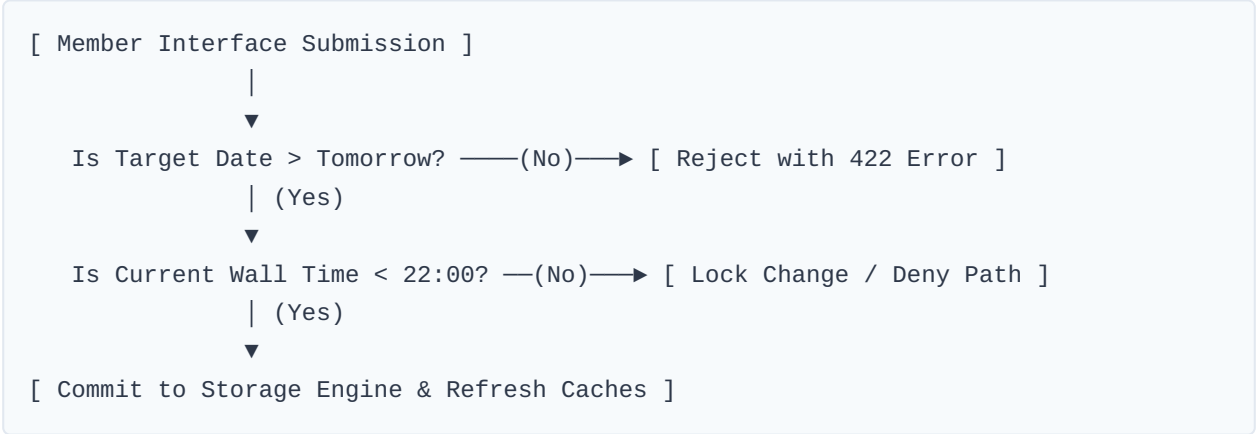
Precise portion allocations require rigid state machine logic to prevent discrepancies between purchased fresh goods and the meal counts updated post-hoc by individual residents.

6.1 The 10:00 PM Strict Cut-Off Window Routine

The meal modification mechanism relies on a fixed scheduling restriction engine. For any calendar target T, updates to meal configurations must occur prior to 10:00 PM ($22:00$ Hours) on day $T-1$.

6.2 State Lifecycle Schema Transition Blueprint

The state pipeline maps alterations chronologically across the following pipeline execution states:



6.3 Code Validation Mechanism Sample

```
const checkCutoff = (targetDateString) => {
    const now = new Date();
    const cutoffTime = new Date();
    cutoffTime.setHours(22, 0, 0, 0); // Lock boundary at 10:00 PM

    const targetDate = new Date(targetDateString);
    const tomorrow = new Date();
    tomorrow.setDate(tomorrow.getDate() + 1);

    if(targetDate.toDateString() === tomorrow.toDateString() && now > cutoffTime) {
        throw new Error("Target timeline constraint violation. Changes are
locked.");
```

```
    }
};
```

7. AUTOMATED MONTHLY FINANCIAL RECONCILIATION ENGINE

At the exact closing moment of a monthly billing iteration cycle, the system runs an automated reconciliation function to determine the structural net financial balances of the entire house without manual ledger cross-referencing.

7.1 Mathematical Optimization Formulation

The algorithmic sequence determines execution states through consecutive stages:

First, calculation of the standard operational base cost rate per consumer unit allocation:

```
Meal Rate (R) = (Total Month Outlays) / (Total Aggregate Consumer Counts)
```

Next, calculating individual tracking statuses and assigning reimbursement balances:

```
Personal Liability (L) = (User Aggregate Meals * R) + Specified Personal Adjustments
Net Final Delta (Δ) = User Total Verified Deposits - Personal Liability (L)
```

7.2 Finalization Matrix Output Structure

Resident Name	Total Meals	Deposited Capital	Computed Cost Liability	Final Action Ledger Status
Alex Smith	62.0	$150.00	$124.00	**Payout Refund: + $26.00**
John Doe	58.5	$100.00	$117.00	**Deficit Owed: -$17.00**

8. SERVICE INTEGRATION, SECURITY & AUDIT PARADIGMS

8.1 Digital Financial Gateways & Verification Webhooks

To prevent falsified transaction logs, incoming deposit requests using Mobile Financial Services (MFS) require verified webhook loop handlers backed by cryptographic signatures.

```
// Transaction webhook authentication verification pattern
app.post('/api/v1/hooks/payments', async (req, res) => {
    const signature = req.headers['x-mfs-signature'];
    if (!verifySignature(req.body, signature, process.env.WEBHOOK_SECRET)) {
        return res.status(401).send("Invalid Webhook Source Execution");
    }
    await processDepositLog(req.body);
    res.status(200).send({ status: "SUCCESS" });
});
```

8.2 Security Guardrails & Cryptographic Log Tracing

Any administrative modifications targeting underlying financial data paths write an immutable record directly into a tracking schema block, preventing manual internal modifications or tampering:

Timestamp Event	Actor ID	Operation Mode	Entity Altered	Snapshot Delta Values
2026-06-30 09:12 UTC	User #04 (Manager)	UPDATE [Override]	Meal Log Day #12	Changed meal value from [0.0] to [1.0]
2026-06-30 14:02 UTC	User #01 (Admin)	APPROVE_CASH	Deposit Ref #981	Marked status tracking to [CLEARED]

[END OF TECHNICAL BLUEPRINT SPECIFICATION DOCUMENT - VERSION 2.0.0]